ML–SIC–ATP Theory

Learned Search Policies for Proof Search

What Machine Learning Can and Cannot Buy an Automated Theorem Prover, with a Labelled Theorem Graph Built from Proof Data

Version 001

Brian Tenneson
M.S., Applied Data Science
Currently Unaffiliated
email: btenneson2301@baypath.edu

July 27, 2026

ML-SIC-ATP-Theory-Brian_Tenneson_001 | Version 001 | July 27, 2026
Companion program: `ml_sic_atp.py`

https://btenneson2301.substack.com

Abstract

A trained proof-search heuristic is a *nondeterministic proof-search policy*: a function from a proof state to a set of candidate one-step extensions. Neural premise selectors, tactic predictors, language models emitting proof steps and learned value functions all have this form, and identifying them as such places them inside a framework where several questions about them are already settled.

Three consequences follow. There is a hard floor: no policy, however trained, produces a target formula earlier than its closure depth, so learning reduces the *breadth* of proof search and never its *depth*. There is an exact account of what pruning costs: proof-covering policies retain completeness and reach the shortest-proof horizon, while a three-formula example shows a greedy policy can fail to reach provable targets at all. And there is a correct question in place of a tempting but vacuous one: since a geodesic in an unweighted graph is by definition a shortest path, asking whether shortest proofs are geodesics claims nothing, whereas asking whether an *admissible* heuristic can be learned is substantive, because only admissibility preserves the optimality guarantee of A* search.

The paper is careful about one structural point that is easy to get wrong. A theorem graph is not automatically acyclic. The consequence relation of a formal system may return to its own axiom — in Hofstadter's MIU system, MI does so in five steps — so acyclicity must be established rather than assumed. It is established here for the graph as recorded, and the depth computation is arranged not to depend on it.

A companion program builds labelled theorem graphs from two sources: proofs generated by running the canonical rule-enumeration machine on a propositional Hilbert system, where

closure depth and proof length are ground truth, and real Metamath databases, whose proofs are already dependency lists and so require no proof reconstruction. On 985 generated statements the bound $\delta_F(\Gamma, \varphi) + 1 \leq \|\varphi\|_F(\Gamma)$ holds without exception; an oracle path to a depth-6 target touches under one percent of what unguided closure materializes; and a premise-selection policy trained under a temporal split reaches MRR 0.60 against 0.32 for a frequency baseline. On Metamath data the bound is exceeded in a controlled way, which is not an error: it measures lemma reuse, since a Metamath step may cite an earlier theorem rather than re-derive it and so records a proof length in a compiled system.

Keywords: automated theorem proving, machine learning, proof search, search policy, premise selection, closure depth, proof horizon, admissible heuristic, A* search, theorem graph, acyclicity, Metamath, benchmark contamination, algorithm selection.

Contents

1 Introduction

Machine learning improves proof search. That much is settled empirically and needs no defence here; [12] surveys the field. The question this paper takes up is narrower and, so far as I can tell, more useful: *what exactly does it improve, and what can it not touch?*

The formal-system vocabulary used throughout — $F = (x, y, z)$, the consequence operator Con_F, semi-ideal computers and the canonical rule-enumeration machine — is that of [1], whose Sections 2, 5, 6 and 7 supply the results invoked below.

The answer turns on an identification that costs nothing and yields a good deal. A trained model, whatever its architecture, is consulted at a proof state and returns candidate next steps. That is precisely a search policy in the sense formalized below. Once the identification is made, results proved for arbitrary policies apply to learned ones without modification, and they are sharper than the informal claims usually made on learning's behalf.

Sections 2 and 3 set up the graph and prove the limits. Section 4 replaces a vacuous question with a well-posed one. Sections 5–7 state what is empirical rather than proved, describe an architecture, and give an evaluation protocol. Section 8 reports measurements from a working program.

2 The labelled theorem graph

Definition 2.1 (Consequence)**.** Let $F = (x, y, z)$ be a formal system and $\Gamma \subseteq y$ a hypothesis set. Write β is a *direct consequence* of $A \subseteq y$ if there are k, a rule $r \in z_k(F)$ and a tuple $(a_1, \ldots, a_k) \in A^k$ with $r(a_1, \ldots, a_k) = \beta$. Put $D_F(A) := A \cup \{\beta : \beta \text{ a direct consequence of } A\}$, $D_F^0(\Gamma) := \Gamma$ and $D_F^{m+1}(\Gamma) := D_F(D_F^m(\Gamma))$. The consequence set is $\mathrm{Con}_F(\Gamma) = \bigcup_m D_F^m(\Gamma)$. There is no consequence set of F alone: consequences are always relative to Γ.

Definition 2.2 (Labelled theorem graph)**.** The *formula-level theorem graph* of F has the elements of y as vertices, and an edge $a \to \beta$ labelled r whenever $\beta = r(a_1, \ldots, a_k)$ for some rule r and some tuple containing a. The *state theorem graph* for Γ has as vertices the proof states $p \subseteq y$ with $\Gamma \subseteq p$, and an edge $p \to p \cup \{r(a_1, \ldots, a_k)\}$ labelled $(r, a_1, \ldots, a_k)$ whenever each $a_i \in p$ and the value is defined.

2.1 Acyclicity is a claim requiring proof

It is tempting to treat a theorem graph as a directed acyclic graph, on the intuition that proving moves forward. The intuition is wrong for the formula-level graph, and the counterexample is not exotic.

Example 2.3 (MIU returns to its own axiom)**.** Hofstadter's MIU system [3] has axiom MI and rules

$$\text{(I) } x\mathrm{I} \to x\mathrm{IU}, \qquad \text{(II) } \mathrm{M}x \to \mathrm{M}xx, \qquad \text{(III) } x\mathrm{III}y \to x\mathrm{U}y, \qquad \text{(IV) } x\mathrm{UU}y \to xy.$$

Then

$$\mathrm{MI} \xrightarrow{\text{II}} \mathrm{MII} \xrightarrow{\text{II}} \mathrm{MIII} \xrightarrow{\text{I}} \mathrm{MIIIU} \xrightarrow{\text{III}} \mathrm{MIUU} \xrightarrow{\text{IV}} \mathrm{MI},$$

a cycle of length five through the axiom itself. A shorter cycle elsewhere in the system is $\mathrm{MUU} \to \mathrm{MUUUU} \to \mathrm{MUU}$, by rule II with $x = \mathrm{UU}$ and then rule IV. So the formula-level theorem graph of MIU is not acyclic, and neither is that of any system with a reversible rule pair — double negation, commutativity conversions, or introduction followed by elimination.

Two acyclicity claims are nevertheless available, and each is worth stating with its proof, because the computations later depend on one of them.

Proposition 2.4 (Novelty edges in the state graph)**.** *Restrict the state theorem graph to edges $p \to q$ with $q \neq p$. The result is acyclic.*

Proof. Along such an edge $q = p \cup \{\beta\}$ with $\beta \notin p$, so $p \subsetneq q$ and the state strictly increases under inclusion. A directed cycle would give a finite chain of strict inclusions returning to its start, which is impossible. □

Proposition 2.5 (The recorded derivation graph)**.** *Suppose a graph is built by recording each statement once, at its first derivation, with an integer* ord$(\cdot)$ *assigned in the order of recording, and with every premise of a statement already recorded at that time. Then every edge runs from smaller to larger* ord*, and the graph is acyclic.*

Proof. If a is a premise of β then a was recorded before β, so $\mathrm{ord}(a) < \mathrm{ord}(\beta)$. A directed cycle would give $\mathrm{ord}(v) < \mathrm{ord}(v)$ for a vertex v on it. □

Remark 2.6 (What this licenses, and what it does not)*.* Proposition 2.5 is what makes topological sorting legitimate on the graphs the companion program constructs: both the generated data and a parsed Metamath database satisfy its hypothesis, the latter because a Metamath proof may cite only earlier theorems. It licenses nothing about the underlying relation, which Example 2.3 shows may be cyclic. Accordingly the depth computation of Definition 3.1 is arranged as a fixpoint iteration rather than a pass over a topological order, so that it remains correct on cyclic input; a statement reachable only around a cycle receives infinite depth, which is the right answer, since it is never in fact derived.

2.2 Policies

Definition 2.7 (Search policy)**.** An *admissible one-step extension* at $p \subseteq Y$ is a finite set $\Delta \subseteq Y$ every member of which is a direct consequence of p. A *nondeterministic proof-search policy* on Y assigns to each pair (p, m) a nonempty set $\Sigma(p, m)$ of admissible one-step extensions at p. A *branch* from Γ is a sequence $\langle p_1, p_2, \dots \rangle$ with $p_1 = \Gamma$ and $p_{m+1} = p_m \cup \Delta_m$ for some $\Delta_m \in \Sigma(p_m, m)$.

Remark 2.8 (A trained model is a policy)*.* A neural premise selector, a tactic-prediction model, a language model emitting proof steps and a learned value function guiding best-first search are all functions from a proof state to a set of candidate one-step extensions. The learned parameters determine which extensions are offered and in what order; they do not change what an extension is. A learned prover is therefore a pair (policy, execution strategy), and every result below applies to it whatever its architecture and however it was trained.

3 What is provable

Definition 3.1 (Closure depth)**.** $\delta_F(\Gamma, \varphi) := \min\{m \geq 0 : \varphi \in D_F^m(\Gamma)\}$, and ∞ if no such m exists.

Theorem 3.2 (Depth floor for every policy)**.** *Let Σ be any policy on Y, $\Gamma \subseteq Y$, and let $b = \langle p_1, p_2, \dots \rangle$ be any branch from Γ. Then $p_m \subseteq D_F^{m-1}(\Gamma)$ for every $m \in \mathbf{N}^+$. Consequently $\varphi \in p_m$ implies $m \geq \delta_F(\Gamma, \varphi) + 1$: no policy produces φ at a stage earlier than its closure depth.*

Proof. Induction on m. For $m = 1$, $p_1 = \Gamma = D_F^0(\Gamma)$. Assume $p_m \subseteq D_F^{m-1}(\Gamma)$. Every $\beta \in \Delta_m$ is a direct consequence of p_m, hence of a subset of $D_F^{m-1}(\Gamma)$, so $\beta \in D_F(D_F^{m-1}(\Gamma)) = D_F^m(\Gamma)$; and $p_m \subseteq D_F^{m-1}(\Gamma) \subseteq D_F^m(\Gamma)$. Hence $p_{m+1} = p_m \cup \Delta_m \subseteq D_F^m(\Gamma)$. If $\varphi \in p_m \subseteq D_F^{m-1}(\Gamma)$ then $\delta_F(\Gamma, \varphi) \leq m - 1$. □

Remark 3.3 (Learning reduces breadth, not depth). The bound holds for *every* policy and is proved from the definition of an admissible extension alone, so no architecture, data volume or training signal affects it. What a policy controls is $|p_m|$: how much of $D_F^{m-1}(\Gamma)$ it actually materializes on the way. Unguided closure materializes all of it; a good policy materializes a thin subset containing the target.

Proposition 3.4 (The quantity learning acts on). *For any branch, $\Gamma \subseteq p_m \subseteq D_F^{m-1}(\Gamma)$, and both inclusions can be strict. The unguided canonical machine, with $C(\Gamma, m) = D_F^{m-1}(\Gamma)$, attains the upper bound; it arises as a branch exactly when F is finitely branching on Y, since Definition 2.7 requires extensions to be finite. The improvement available to a policy is the ratio $|p_m|/|D_F^{m-1}(\Gamma)|$, and experimental claims of speedup should report it.*

Proposition 3.5 (Closure depth is below proof length). *If $\Gamma \vdash_F \varphi$ then $\delta_F(\Gamma, \varphi) + 1 \leq \|\varphi\|_F(\Gamma)$, where $\|\varphi\|_F(\Gamma)$ is the length of a shortest linear proof.*

Proof. Let $\pi = \langle \pi(1), \ldots, \pi(n) \rangle$ be a proof from Γ. By induction on j, $\pi(j) \in D_F^{j-1}(\Gamma)$: the only admissible first line is a hypothesis, since every rule needs a premise, so $\pi(1) \in \Gamma = D_F^0(\Gamma)$; and $\pi(j)$ is either in Γ or a direct consequence of $\{\pi(1), \ldots, \pi(j-1)\} \subseteq D_F^{j-2}(\Gamma)$, hence lies in $D_F^{j-1}(\Gamma)$. Take $j = n$ and minimize over proofs. □

Theorem 3.6 (Branch covering). *Call Σ* proof-covering *if for every Γ and every proof π from Γ there is a branch with $\{\pi(1), \ldots, \pi(j)\} \subseteq p_j$ for all $j \leq |\pi|$. If Σ is proof-covering and $\Gamma \vdash_F \varphi$, some branch reaches φ by stage $\|\varphi\|_F(\Gamma)$.*

Proof. Take a shortest proof π of φ, of length n. Covering supplies a branch with $\{\pi(1), \ldots, \pi(j)\} \subseteq p_j$, so $\varphi = \pi(n) \in p_n$. □

Remark 3.7 (Floor below ceiling, and the gap is the working room). Proposition 3.5 places the floor of Theorem 3.2 below the ceiling of Theorem 3.6, so the two are consistent. The gap between them is where a policy operates, and it is the cost of serializing a parallel derivation: D_F fires every rule at once, while a linear proof must list its premises one at a time.

3.1 What pruning costs

Example 3.8 (A greedy policy loses completeness). Let $y = \{a, b, c\}$ with unary rules $r(a) = b$ and $s(a) = c$, let $\Gamma = \{a\}$ and let the target be c. Put $\Sigma(p, m) := \{\{b\}\}$ whenever $a \in p$ and $\{\varnothing\}$ otherwise. This is a legitimate policy: $\{b\}$ is an admissible one-step extension at any state containing a. Its unique branch is $\{a\}, \{a, b\}, \{a, b\}, \ldots$, which never contains c, although $\Gamma \vdash_F c$ in one rule application.

Remark 3.9 (The central practical risk). Example 3.8 is the formal shadow of a system that commits to its top-ranked tactic without backtracking. The policy is not defective; it is simply not proof-covering, so Theorem 3.6 does not apply to it. A learned model ranks successors, and the ranking is only as good as its training distribution; on inputs unlike the corpus the top-ranked successor can

be systematically wrong. Hence the engineering rule: *the model supplies the ordering, the wrapper supplies the guarantee.* A research programme on learned proof search must specify both.

3.2 Lemma libraries

Theorem 3.10 (Conservative compilation)**.** *Let $\mathcal{T} = \{(\Gamma_i, \varphi_i) : 1 \le i \le N\}$ be finite with $\Gamma_i \vdash_F \varphi_i$ for each i. There is a finite set D of derived rules such that the canonical target-search machine over $F^D = (x, y, z \cup D)$ reaches φ_i from Γ_i within two stages for every i; and every theorem of F^D is a theorem of F.*

Remark 3.11 (Premise selection is compilation with a learned index). A retrieval-based premise selector over a large library does two things: the library supplies D, and the learned component supplies an index into it — a guess at which few of the available lemmas bear on the current goal. Theorem 3.10 says the horizon collapses once the right derived rule is available, so the learning problem is retrieval, not horizon reduction. This predicts the observed shape of results: large gains where the library already contains what is needed, small gains where genuinely new intermediate structure is required, for which no D assembled from existing lemmas suffices.

4 Admissible heuristics

Remark 4.1 (A question that claims nothing). One might conjecture that shortest proofs correspond to geodesics under a suitable theorem-space metric. In the state theorem graph with unit edge costs, the distance from Γ to the nearest state containing φ *is* the least number of one-formula expansions required, and a geodesic is by definition a path realizing the distance. The conjecture asserts that shortest paths are shortest paths.

Definition 4.2 (Admissible, consistent)**.** Let $d^*(p, \varphi)$ be the true distance from p to the nearest state containing φ. A heuristic h is *admissible* if $h(p, \varphi) \le d^*(p, \varphi)$ for all p, and *consistent* if $h(p, \varphi) \le 1 + h(q, \varphi)$ for every edge $p \to q$.

Proposition 4.3 (What admissibility buys and costs)**.** *Assume the reachable portion of the graph is locally finite and effectively enumerable. If h is admissible, A* with cost $g(p) + h(p, \varphi)$ returns a proof of least length; if h is consistent, no state is expanded twice. If h overestimates anywhere on an optimal path, the returned proof may be longer than the shortest and the optimality guarantee is lost.*

Proof. The standard optimality theorem for A* [2, 7], applied to a unit-cost graph; local finiteness is what makes the open list well defined at each step. □

Remark 4.4 (The substantive question). Learned heuristics are trained to minimize prediction error — squared error, or a ranking loss against observed proof lengths. Nothing in that objective encourages under-estimation, so a model with small average error overestimates on roughly half its inputs and is essentially never admissible. Two well-posed questions follow:

1. Can an admissible heuristic be learned — by training against a lower bound on remaining distance, or by shifting a trained model down by a certified error bound?
2. If not, at what cost? For ε-admissible h, with $h \le (1 + \varepsilon)d^*$, A* returns a proof within $1 + \varepsilon$ of shortest. Can a model be certified ε-admissible on a distribution, and what ε is achievable?

5 Hypotheses

The following are empirical claims about learned models. None is a consequence of any definition above, and each is stated so that it can fail.

Hypothesis 5.1 (Local structure is learnable)**.** There is a learnable embedding of formulas under which co-occurrence in proofs is predictable: a retrieval model trained on a proof corpus selects relevant premises with precision substantially above a frequency baseline.

Hypothesis 5.2 (Embedding stability)**.** Embeddings trained on disjoint proof corpora agree, up to an affine transformation, on the relative similarity of formulas occurring in both. Testable by fixing a shared subset, training separately, and measuring rank correlation of pairwise similarities.

Hypothesis 5.3 (Solver specialization)**.** Distinct ATP systems have distinguishable competence profiles: there is a learnable map from goal features to the solver most likely to succeed, and a portfolio using it beats any single constituent.

This is the *algorithm selection* problem [9], with a substantial literature: portfolio solvers have been standard in satisfiability competitions for two decades [10], and Isabelle's Sledgehammer has dispatched goals to a portfolio of external provers since before the neural era [11]. The contribution available is not the existence of specialization but its structure — whether competence is predictable from cheap syntactic features, and whether the partition is stable across libraries.

Hypothesis 5.4 (Cross-library transfer)**.** A policy trained on one formal library retains a substantial fraction of its advantage on a library with a different logical foundation. Untestable without the controls of Section 7.

6 Architecture

The design that currently leads benchmarks is:

1. a language model proposes a candidate proof, or a next tactic, in the concrete syntax of a proof assistant;
2. the proof assistant *verifies* it, giving ground truth with no learned critic and no possibility of a hallucinated proof being accepted;
3. successful traces are added to training data and the model is updated, typically by reinforcement learning;
4. search — best-first over tactic candidates, or sampling many whole proofs — wraps the model to restore the completeness that Example 3.8 shows a greedy policy lacks.

The verifier is the essential component. It makes the training signal trustworthy and the enterprise safe: an unsound proof is rejected mechanically, so the learned component may be arbitrarily unreliable without compromising correctness. In the vocabulary above, the model supplies Σ and the assistant guarantees that every accepted branch is rule-driven.

Remark 6.1 (Build the solver selector first). Of the components available, the solver selector of Hypothesis 5.3 is the one to build first: a well-defined supervised problem, established prior art, measurable gains, and independent of every contested claim in this paper.

7 Evaluation

The dominant failure mode in this area is measuring memorization.

Remark 7.1 (Contamination). Models are trained on corpora overlapping the benchmarks, so a system trained on a library and evaluated on problems derived from it may be retrieving rather than proving. The field has responded with perturbation benchmarks, on which models lose accuracy when statements are altered in meaning-preserving ways, and with evaluation sets drawn from competitions held after the training cutoff.

Remark 7.2 (Benchmarks with headroom). MiniF2F, 488 competition problems formalized across several systems [6], is close to saturated: leading provers exceed ninety percent on its test split, so differences between strong systems sit within noise and largely reflect contamination. PutnamBench, 1692 formalizations of 640 Putnam problems in Lean, Coq and Isabelle [8], remains discriminating: reported results on its Lean subset of 672 problems are on the order of ninety solved. Figures across target languages are not comparable, so the subset should be named whenever a number is quoted.

A defensible protocol:

1. *Temporal split.* Train on library commits predating a fixed date; evaluate on theorems added after. This is the only split approximating the deployment condition.
2. *Perturbation control.* Report accuracy on meaning-preserving rewrites alongside originals; a large gap indicates memorization.
3. *Compute-matched baselines.* Report the unguided baseline at equal wall-clock and equal node expansions. A policy that wins on wall clock but loses on expansions bought its gain with hardware.
4. *Breadth ratio.* By Proposition 3.4 the improvable quantity is $|p_m|/|D_F^{m-1}(\Gamma)|$; reporting it separates search reduction from faster iteration.

8 Measurements

The companion program `ml_sic_atp.py` implements the pipeline. It depends on `numpy` alone — graph algorithms and logistic regression are written out — so it runs on a bare Python installation.

8.1 Choosing a corpus

The requirement is real proofs convertible to a labelled theorem graph without proof reconstruction. That selects **Metamath** nearly uniquely.

Remark 8.1 (Why Metamath). A Metamath proof [5] is a list of label references: `thm $p - ... = L1L2....|` names exactly the axioms and earlier theorems it invokes. The dependency edges are written in the file, so building the graph is parsing rather than inference. Extracting the same information from Lean's Mathlib [4] or Isabelle's Archive of Formal Proofs requires running the toolchain and instrumenting elaboration. Metamath also has no tactic layer, so nothing hides inside automation — the property the depth measurement needs. And because proofs cite only earlier theorems, a parsed database satisfies the hypothesis of Proposition 2.5.

Databases, smallest first: `ql.mm` (quantum logic), `nf.mm` (New Foundations), `iset.mm` (intuitionistic set theory), `set.mm` (classical ZFC, about 43,900 proved theorems). For a first run `iset.mm` is the recommended compromise — real mathematics, a fraction of the size. Each is a single text file:

```
curl -O https://raw.githubusercontent.com/metamath/set.mm/develop/iset.mm
```

A real corpus cannot supply ground truth for closure depth in the base system, because human-written proofs cite lemmas. The program therefore also generates its own data.

8.2 Generated data

In `-mode synth` the program executes $C(\Gamma, 1) = \Gamma$, $C(\Gamma, m+1) = D_F(C(\Gamma, m))$ on a propositional Hilbert system: Γ is the atom set and the rules are modus ponens together with the axiom schemes read as generating rules, instantiating on formulas already derived. A size bound keeps Y finite. Each new formula records the rule and the exact premises that produced it, so δ and $\|\cdot\|$ are ground truth and both are measured in the same system — which is what makes Proposition 3.5 testable.

Remark 8.2 (A modelling point found by running it). Instantiating the schemes only at atoms makes the consequence set saturate after one round: every instance is an implication whose antecedent is not separately available, so modus ponens rarely detaches. Treating the schemes as rules instantiating on derived formulas is both faithful to how Hilbert systems are used and necessary for $\mathrm{Con}_F(\Gamma)$ to have depth. Stage sizes: $3, 39, 111, 381, 579, 789, 985$.

8.3 Results

Labelled theorem graph (generated, 7 stages)	
vertices / edges	985 / 1991
hypotheses in Γ / derived	3 / 982
maximum closure depth	6
stage sizes $\lvert D_F^{m-1}(\Gamma)\rvert$	3, 39, 111, 381, 579, 789, 985
Test of $\delta_F(\Gamma,\varphi)+1 \le \lVert\varphi\rVert_F(\Gamma)$	
statements checked / violations	985 / **0**
slack: mean / median / max	2.26 / 2.0 / 4
Oracle breadth ratio, deepest targets	
statements on an oracle path to depth 6	7–8
materialized by unguided closure	985
ratio	0.007–0.008
Premise selection, temporal split	
training theorems / pairs	687 / 14,735
held-out theorems	295
recall@1 learned / baseline	0.190 / 0.037
recall@5 learned / baseline	0.514 / 0.334
recall@10 learned / baseline	0.592 / 0.451
MRR learned / baseline	**0.599** / 0.322

8.4 Reading the numbers

Remark 8.3 (The bound holds; the slack is the working room). No statement among the 985 violates $\delta_F(\Gamma,\varphi)+1 \le \lVert\varphi\rVert_F(\Gamma)$, as Proposition 3.5 requires — which confirms that the implementation measures what it claims, rather than establishing anything new. The informative quantity is the

slack, mean 2.26: a depth-d statement takes about $d+3$ lines, because a linear proof serializes what D_F derived in parallel.

Remark 8.4 (The breadth ratio is an oracle bound). Reaching a depth-6 target requires 7 or 8 statements on a path, against 985 materialized by unguided closure. This ratio is computed from the ancestor set of the target — that is, with hindsight — so it is what a *perfect* policy would achieve, and is the ceiling on available leverage rather than an achieved result. No learned policy attained it here. The floor it cannot move is the depth: 6 stages, whatever the policy.

Remark 8.5 (The learned policy, honestly). Under a temporal split the learned ranker reaches MRR 0.599 against 0.322 for frequency ranking. Eight cheap syntactic features under logistic regression were used, to test Hypothesis 5.1 rather than to compete with a neural prover.

The features are standardised before fitting, and this is not a detail. Their natural ranges differ by more than an order of magnitude — the recency term is (goal order − candidate order)/1000, which on a corpus the size of `set.mm` reaches about 44 while every other feature stays inside $[0, 1]$ — so an unstandardised fit is dominated by whichever feature happens to carry the largest units, and reports the size of the corpus rather than the structure of the data. Standardising moved MRR from 0.53 to 0.60 and recall@10 from 0.48 to 0.59 on the same data and the same features.

Three caveats bound what this shows: the corpus is small and generated from three rules over three atoms, so its regularity far exceeds real mathematics; the baseline is weak; and the margin narrows as k grows, consistent with the model learning to rank relevant premises above merely popular ones rather than finding premises the baseline misses entirely. It is evidence that the pipeline measures something, not a result about theorem proving.

Remark 8.6 (Metamath exceeds the bound, and that is informative). Run against a Metamath database, the same check reports statements whose proof length falls below their closure depth. These are not errors. A Metamath step may cite an earlier theorem instead of re-deriving it, so the step count is a proof length in F^D — the system augmented by every previously proved theorem as a derived rule — while the dependency depth is measured in F. Comparing them measures Theorem 3.10 rather than testing Proposition 3.5, and the program labels such cases as compilation events with the steps saved. The methodological lesson: any corpus of human-written proofs is a corpus of proofs in F^D, and which system a length lives in must be tracked explicitly.

8.5 The cyclic case as a test

Running `-mode miu` builds the MIU system of Example 2.3 and confirms the cycle computationally: eight statements of length at most ten lie on cycles, among them MI itself. The mode exists to keep Section 2 honest — it is the case that would break any procedure assuming acyclicity, and it verifies that the fixpoint depth computation does not.

8.6 Running it

```
# generated data, exact ground truth, no network needed
python3 ml_sic_atp.py --mode synth --depth 6 --cap 3000 --json out.json

# the cyclic counterexample
python3 ml_sic_atp.py --mode miu

# a real corpus
curl -O https://raw.githubusercontent.com/metamath/set.mm/develop/iset.mm
python3 ml_sic_atp.py --mode metamath --db iset.mm --limit 6000
```

8.7 Scope of these measurements

This is a first run on a small generated corpus together with a parser demonstration. It is evidence that the pipeline measures what it claims, not a result about theorem proving. The generated system is propositional and tiny beside `set.mm`; the features are deliberately crude; no perturbation control was applied, since generated data cannot be contaminated; and no comparison against a real prover was attempted. Running the same pipeline on `iset.mm` or `set.mm` is what would make the premise-selection number, and the compilation-event count, mean something about practice.

9 Falsifiable predictions

Prediction 9.1. No learned system will produce a proof at a stage below its closure depth, in any architecture. By Theorem 3.2 a violation would indicate an implementation error — most likely a hidden lemma library, which changes F and hence D_F.

Prediction 9.2. Systems following a learned policy greedily will show systematic incompleteness on goals whose shortest proof begins with a low-ranked step, and restoring search will recover them. This is Example 3.8 at scale.

Prediction 9.3. Gains from premise selection will correlate with the availability of a sufficient lemma set, and fall sharply on goals needing intermediate structure absent from the library, because by Theorem 3.10 the compilation effect is bounded by what D contains.

Prediction 9.4. Off-the-shelf learned distance estimators will be inadmissible on a measurable fraction of states, and A* using them will return non-shortest proofs at a rate increasing with that fraction.

10 Programme

Phase 0. Build the evaluation harness before training anything: temporal split, perturbation set, compute-matched unguided baseline, and instrumentation for the breadth ratio. Nothing measured before this exists is interpretable.
Phase I. The solver selector of Hypothesis 5.3. Smallest scope, established prior art, immediate payoff, independent of every contested claim here.
Phase II. Premise selection, evaluated as retrieval — precision at k against the premises actually used — so that retrieval quality is separated from the downstream prover.
Phase III. Test Hypothesis 5.2 directly: disjoint corpora, shared formula subset, rank correlation of similarities.
Phase IV. Admissibility (Remark 4.4): can a distance estimator be trained or shifted to be admissible, and at what cost in tightness?
Phase V. Cross-library transfer, meaningful only once Phases 0–III establish that the within-library effect is real.

11 Conclusion

That machine learning helps proof search needs no defence. What it helps with admits a precise answer. A trained model is a search policy; policies are bound below by closure depth and above,

when proof-covering, by the shortest-proof horizon; the gap between those bounds is the serialization cost of a parallel derivation, and it is where all the available leverage lives. Pruning buys speed at the price of completeness unless a search wrapper restores it. Lemma libraries buy horizon collapse, and premise selection is an index into that library rather than a reduction of search depth. And the question worth asking about learned distance estimates is not whether shortest proofs are geodesics — they are, trivially — but whether admissibility can be learned, since only admissibility preserves optimality.

These are modest results. They have the advantage of following from definitions rather than from a metaphor, and of being accompanied by a program that measures them.

Acknowledgements

The author thanks AI assistants used as editorial and verification tools during preparation, including for the identification of the MIU counterexample of Example 2.3, which established that acyclicity of a theorem graph requires proof rather than assumption. All mathematical claims, interpretations and final decisions remain the responsibility of the author.

References

[1] Tenneson, B. (2026). *Depths of a Simulation and Formalized Self-Awareness*, Merged Edition. Part I, Sections 2, 5, 6 and 7.

[2] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.

[3] Hofstadter, D. R. (1979). *Gödel, Escher, Bach: An Eternal Golden Braid*. Basic Books. (The MIU system.)

[4] The mathlib Community (2020). The Lean mathematical library. *CPP*.

[5] Megill, N. D., & Wheeler, D. A. (2019). *Metamath: A Computer Language for Mathematical Proofs* (2nd ed.). Lulu Press.

[6] Zheng, K., Han, J. M., & Polu, S. (2022). MiniF2F: A cross-system benchmark for formal Olympiad-level mathematics. *ICLR*.

[7] Pearl, J. (1984). *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley.

[8] Tsoukalas, G., et al. (2024). PutnamBench: Evaluating neural theorem-provers on the Putnam Mathematical Competition. *NeurIPS Datasets and Benchmarks*.

[9] Rice, J. R. (1976). The algorithm selection problem. *Advances in Computers*, 15, 65–118.

[10] Xu, L., Hutter, F., Hoos, H. H., & Leyton-Brown, K. (2008). SATzilla: Portfolio-based algorithm selection for SAT. *Journal of Artificial Intelligence Research*, 32, 565–606.

[11] Paulson, L. C., & Blanchette, J. C. (2010). Three years of experience with Sledgehammer, a practical link between automatic and interactive theorem provers. *IWIL*.

[12] Li, Z., Sun, J., Murphy, L., Su, Q., Li, Z., Zhang, X., Yang, K., & Si, X. (2024). A survey on deep learning for theorem proving. *Conference on Language Modeling*.